SoC Audio Processing

Final Project Report

EEL4930/5934

Case Zumbrum | Cameron Doig-Daniels | Jacob Mack | William Jones

GitHub Link

https://github.com/CaseZumbrum/SoC_Final_Project

Video Demo Link

https://drive.google.com/file/d/198vw9h_tsTv8srPv_WdDUSgBrIq21z0e/view?usp=sharing

Overview

This project implements a real-time guitar effects pedal on the Urbana development board. An analog audio signal enters through the on-board ADC, passes through one of several configurable effects, and exits as a PWM signal routed to the speaker pins. The system is organized into four main subsystems: the ADC audio input, the processor hardware, the FPGA hardware effects, and the software control running on PetaLinux.

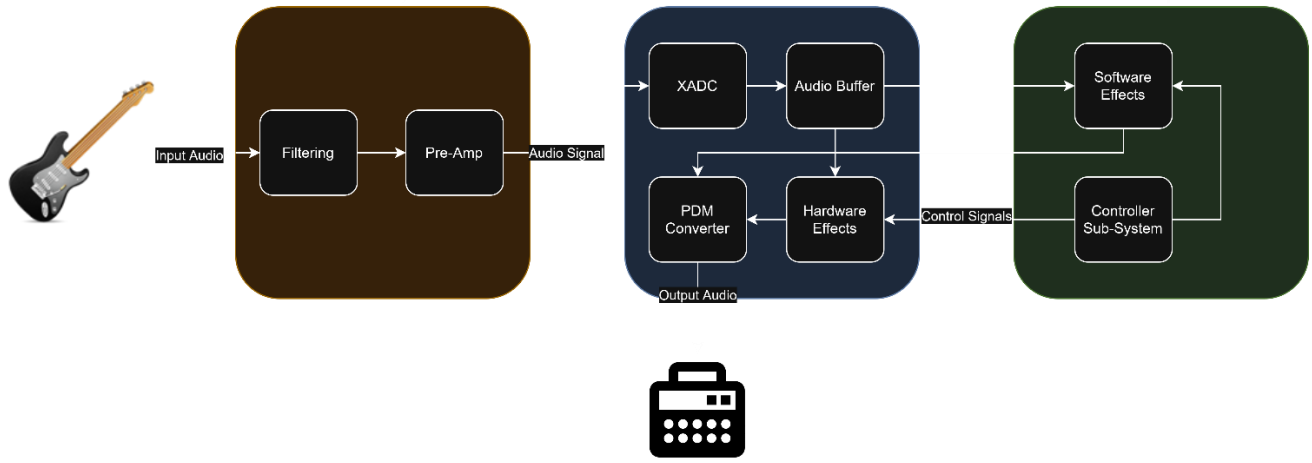

Figure 1: System Design Diagram

1. Guitar Effects Overview

The system implements five distinct audio effects, split between hardware acceleration on the FPGA and software processing on the MicroBlaze.

Passthrough (No Effect)

Passthrough is the baseline mode. The raw ADC sample is transferred directly to the PWM output via DMA with no modification.

Echo (Hardware accelerated)

Echo is a time-based effect that produces a repeat of the original signal about one second delayed. In this implementation, the incoming sample is stored in a circular buffer and retrieved a second later.

This effect was done with a custom memory mapped accelerator that accepts samples and produces samples with echo applied.

Reverb (Hardware accelerated)

Reverb simulates the natural acoustic echoes. This is very similar to the echo effect, except the delay time is much shorter (roughly 100ms) and it applies the echo recursively. This helps create a much fuller sound.

This effect was done with a custom memory mapped accelerator that accepts samples and produces samples with reverb applied.

Hard Clipping

Hard clipping is a form of distortion that sets a strict threshold for the audio signals amplitude. This makes the sound much harsher than a simple passthrough.

Fuzz

Fuzz is another form of distortion that makes the audio more “fuzzy”. In this implementation we clip and scale the audio. This makes the distance between any two samples greater, increasing the “fuzziness” of the audio.

---

2. Audio Input

The Urbana board features an on-board 12 bit ADC system that we have configured to perform continuous differential signals at 48 kHz. These samples are stored in a memory mapped register that is accessible across an AXI interface.

3. Audio Output

The Urbana board has an on-board audio processor that expects a PWM (or PDM) modulated audio signal. This signal is then output across the headphone jack on the board. To interface with this, we used a PWM modulator that can take in audio samples and produce a PWM output whose duty-cycle corresponds to the amplitude of the audio signal at that point. This modulator is memory mapped using an AXI GPIO module. Notably, this PWM signal is much lower fidelity than more commonly used protocols such as I2S.

---

4. Processor Hardware and AXI Fabric

MicroBlaze Processor

We are using a very similar setup to project 3 for the processor. A MicroBlaze soft processor is connected to the DDR memory of the Urbana board, as well as all of the clocks, peripherals, and caches necessary for running PetaLinux.

AXI Peripheral Interconnect

We used the AXI protocol to connect our custom peripherals to the MicroBlaze. This maps all of the hardware effects, modulators, and inputs to a single memory space. We also used an AXI DMA system to allow for direct memory transfers between our custom peripherals.

---

5. Software and OS Layer

PetaLinux

We ran PetaLinux on the Urbana board, using a very similar setup to project 3. The primary difference is the addition of a custom module for audio control in the Petalinux build. Our custom module (audio-application) is used to control the hardware and software effects on the board.

User Space Application

The audio controller uses the mmap library in C to interface with the DMA and peripherals from the user space. We essentially map the true memory addresses (provided at /dev/mem) to a file that we can treat as a pointer. This allows us to directly access the registers of our peripherals as if we were running directly on metal.

The application serves as a simple CLI, allowing users to select which effect they want to use before configuring the hardware and software to perform that effect.

Summary

Our design is a full system-on-chip. Featuring hardware acceleration (hardware effects), an operating system (PetaLinux), and hardware-software interfaces (software effects and memory mapped peripherals).

The use of hardware acceleration allowed for more effects, since the delay and echo require large buffers to store past samples and immediate access to past samples to produce good results. In our testing, pure software implementations of these effects fell flat. However, the ability to also use software effects is very useful for testing and implementing effects that may not be synthesizable.